

LECTURE 02

JavaScript, *the language.*

Forget the framework. Forget the browser. Today: what actually happens when a line of JS runs.

values

scope

closures

async

modules

footguns

WHERE IT RUNS

JS escaped the browser.

2008

Browsers

Chrome's V8 engine made JS fast. Suddenly the language people tolerated became the language people chose.

2009

Node.js

Same engine, on the server. One language, frontend and backend. Career-defining shift.

today

Everywhere

Servers, edge functions, mobile (React Native), desktop (Electron), build tools, even databases.

learn it once. ship it everywhere.

HEADS UP

Looks like Java. Isn't.

Curly braces and semicolons trick you. Underneath, JS is closer to Lisp or Scheme than to Java.

Carry over assumptions and you'll fight the language for years.

approach it fresh. it'll click.

SURPRISES AHEAD

- ✓ Functions are values
- ✓ Objects are mutable bags
- ✓ Variables can hold anything
- ✓ Equality has two flavors
- ✓ this is dynamic
- ✓ Async is single-threaded

THE ATOM

Eight types. That's it.

```
// the seven primitives
typeof 42          // 'number'
typeof "hi"        // 'string'
typeof true        // 'boolean'
typeof undefined   // 'undefined'
typeof null        // 'object' ← bug, kept
typeof Symbol()    // 'symbol'
typeof 10n         // 'bigint'

// ...everything else is an object
typeof []          // 'object'
typeof {}          // 'object'
typeof (x => x)     // 'function'
```

Numbers are all 64-bit floats. $0.1 + 0.2 \neq 0.3$.

Strings are immutable. "a" + "b" makes a new one.

null = "intentionally empty".

undefined = "wasn't set".

VARIABLES

const by default.

```
const name = "ada"    // can't reassign
let count = 0         // can reassign
var old = true        // don't.
```

```
count = 5             // ✓
name = "grace"         // ✗ TypeError
```

```
// const ≠ immutable
const user = { name: "ada" }
user.name = "grace"    // ✓ works!
// the binding is fixed,
// the contents are not.
```

- ✓ const — your default.
- ✓ let — when you must reassign.
- ✗ var — never. legacy.

const fixes the label, not the box.

THE BIG GOTCHA

Primitives copy. Objects share.

```
// primitives – copied
let a = 5
let b = a
b = 10
console.log(a)    // 5 ✓

// objects – shared by reference
const x = { n: 1 }
const y = x
y.n = 99
console.log(x.n)  // 99 ⚠

// to copy, spread it
const z = { ...x } // fresh object
```

This is the source of half the bugs you'll write in your first year.

Especially in React: never mutate state, always make a new object.

"=" with an object means: same box, new label.

FUNCTIONS

Functions are values.

```
// store in a variable
const double = (x) => x * 2

// pass to other functions
[1,2,3].map(double)
// [2, 4, 6]

// return from functions
const multiplier = (n) =>
  (x) => x * n

const triple = multiplier(3)
triple(10)      // 30

// store them in objects, arrays...
// they're just values.
```

This is the single biggest idea separating JS from Java/C++.

Once it clicks, half the language opens up: callbacks, array methods, event handlers, middleware.

if it can be a value, it can be a function.

SCOPE & CLOSURES

Functions remember.

```
function makeCounter() {  
  let count = 0 // "captured"  
  
  return () => {  
    count++  
    return count  
  }  
}  
  
const tick = makeCounter()  
tick() // 1  
tick() // 2  
tick() // 3  
  
// the inner function still has  
// access to 'count' even though  
// makeCounter has long returned.
```

A **closure** is a function bundled with the variables it saw when it was created.

This is how React hooks work. How event handlers remember context. How private state exists in JS.

closures are how JS does memory.

THE CURSED KEYWORD

this — just use arrow functions.

```
const user = {
  name: "ada",
  greet: function() {
    return `hi, ${this.name}`
  }
}
user.greet() // "hi, ada" ✓

const g = user.greet
g() // "hi, undefined" ⚠️

// arrow functions don't have
// their own 'this' — they
// inherit it from outside.
// 99% of the time, that's what
// you want.
```

In a regular function, `this` = whoever called you.

In an arrow function, `this` = whatever it was outside.

modern code: arrows everywhere. problem solved.

DATA STRUCTURES

Objects & arrays.

Object — key/value bag

```
const user = {  
  name: "ada",  
  age: 28,  
  tags: ["admin"]  
}  
  
user.name           // "ada"  
user["name"]        // "ada"  
user.role = "x"     // add key  
delete user.age     // remove
```

Array — ordered list

```
const xs = [1, 2, 3]  
  
xs[0]                // 1  
xs.length            // 3  
xs.push(4)           // mutates!  
[...xs, 4]           // new array  
  
// arrays are objects too  
typeof xs            // 'object'  
Array.isArray(xs)    // true
```

both are mutable. both are passed by reference.

THE DAILY SEVEN

Array methods you'll type 100x a week.

```
const users = [
  { name: "ada",   age: 28 },
  { name: "grace", age: 42 },
  { name: "alan",  age: 35 }
]

users.map(u => u.name)
// ["ada", "grace", "alan"]

users.filter(u => u.age > 30)
// [grace, alan]

users.find(u => u.name === "alan")
users.some(u => u.age < 30)    // true
users.every(u => u.age > 18)   // true

users.reduce((sum, u) => sum + u.age, 0)
```

- ✓ map — transform each
- ✓ filter — keep matching
- ✓ find — first match
- ✓ some / every — check
- ✓ reduce — fold to one value
- ✓ includes — has it?

forget for-loops for transforms.

SUGAR YOU'LL SEE EVERYWHERE

Destructure + spread.

```
// destructuring – pull out values
const { name, age } = user

const [first, second] = arr

// rename + default
const { name: n = "anon" } = user

// in function params
function greet({ name, age }) {
  return `${name} is ${age}`
}
```

```
// spread – copy / merge / inject
const copy = { ...user }
const merged = { ...a, ...b }
const added = { ...user, age: 29 }

// arrays too
const all = [...xs, ...ys, 99]

// rest – collect what's left
const [head, ...tail] = arr
function sum(...nums) { ... }
```

three dots → spread. on the right of "=" → rest.

ES MODULES

Each file is its own little world.

MATH.JS

```
// named exports
export const PI = 3.14159

export function double(x) {
  return x * 2
}

// default export (one per file)
export default function add(a, b) {
  return a + b
}
```

APP.JS

```
// named imports – match the names
import { PI, double } from "./math.js"

// default import – pick any name
import add from "./math.js"

// rename
import { double as x2 } from "./math.js"

// import everything
import * as math from "./math.js"
```

named for many things, default for "the one thing this file is."

SECTION 04

Async.

The thing that trips up every junior.
We're going to take it slow.

THE SETUP

JS does one thing at a time.

Single-threaded. One call stack. No parallelism inside your code.

So how does it handle network requests, timers, file IO?

It hands them off to the runtime (browser or Node) and keeps going.
When the work finishes, your callback gets queued.

that handoff is the whole story.

SYNC VS ASYNC

```
console.log("a")

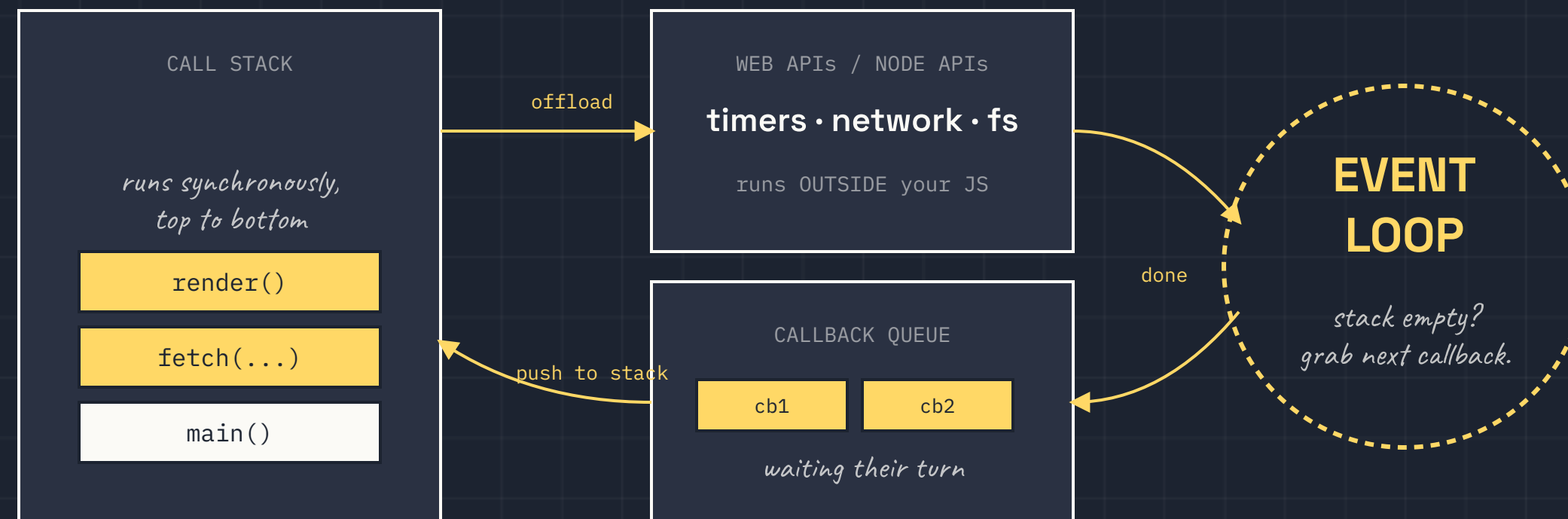
setTimeout(() => {
  console.log("b")
}, 0)

console.log("c")

// prints: a, c, b
// even with delay 0!
```

THE DIAGRAM

The event loop.



PROMISES

A value that doesn't exist yet.

```
const p = fetch("/api/users")
// p is a Promise — pending right now

p.then(res => res.json())
  .then(data => {
    console.log(data)
  })
  .catch(err => {
    console.error(err)
  })
```

A Promise is in one of three states:

- ✓ **pending** — still working
- ✓ **fulfilled** — got a value
- ✓ **rejected** — got an error

Once settled, never changes.

SUGAR ON TOP

async / await — read it like sync code.

.THEN() CHAINS

```
function getUser(id) {  
  return fetch(`/u/${id}`)  
    .then(r => r.json())  
    .then(u => {  
      return fetch(`/p/${u.id}`)  
        .then(r => r.json())  
    })  
    .catch(err => log(err))  
}
```

...SAME CODE WITH AWAIT

```
async function getUser(id) {  
  try {  
    const r1 = await fetch(`/u/${id}`)  
    const u = await r1.json()  
  
    const r2 = await fetch(`/p/${u.id}`)  
    return await r2.json()  
  } catch (err) {  
    log(err)  
  }  
}
```

always wrap awaits in try/catch. always.

COMMON MISTAKES

Three async footguns.

01

Forgetting await

```
const u = getUser()
// u is a Promise!
u.name // undefined ▲
```

02

Awaiting in a loop

```
// slow - one at a time
for (const id of ids) {
  await fetch(id)
}
// fast - parallel
await Promise.all(
  ids.map(fetch)
)
```

03

Unhandled rejection

```
// no try/catch?
// → process.unhandled
// → app crashes
// always handle errors.
```

SECTION 05

Footguns.

Every language has them.
JavaScript has more than most.

EQUALITY

Always ===. Almost never ==.

```
0 == ""           // true   ⚠️
0 == "0"          // true   ⚠️
"" == "0"         // false  ???
false == "0"      // true   ⚠️
null == undefined // true
NaN == NaN        // false  always

// === checks type AND value
0 === ""          // false ✓
0 === 0           // true  ✓
"5" === 5         // false ✓
```

== coerces types — JS quietly converts one side to match the other.

=== compares without coercion.

One useful exception:

```
x == null
```

catches both null and undefined.

COERCION

Memorize the falsy six.

FALSY

```
false
0
"" // empty string
null
undefined
NaN

...and that's it.
```

TRUTHY (EVERYTHING ELSE)

```
"0" // non-empty string!
>false" // also truthy!
[] // empty array – truthy!
{} // empty object – truthy!
-1
Infinity

empty array ≠ falsy. surprises everyone.
```

THE SPOOKIEST BUGS

Don't mutate shared things.

```
// ✗ mutating – bug factory
function addTag(user, tag) {
  user.tags.push(tag)
  return user
}
// caller's user just got changed!

// ✓ immutable – make new things
function addTag(user, tag) {
  return {
    ...user,
    tags: [...user.tags, tag]
  }
}
```

Especially in React: state objects are shared. Mutate one, multiple components break in confusing ways.

Rule of thumb: if you didn't create the object inside this function, don't write to it.

spread, don't push.

RECOGNIZE, DON'T MEMORIZE

JavaScript's greatest hits.

```
typeof null  
// 'object'    – a 1995 bug, never fixed
```

```
NaN === NaN  
// false      – use Number.isNaN()
```

```
[] + []  
// ""         – empty string
```

```
[] + {}  
// "[object Object]"
```

```
0.1 + 0.2  
// 0.30000000000000004
```

```
"5" - 1 "5" + 1  
// 4      "51"    – + is special
```

don't memorize. just recognize when one bites you.

WHAT SURROUNDS THE LANGUAGE

Tools you'll touch this week.

RUN IT

- ✓ `node app.js`
- ✓ browser devtools console
- ✓ `deno` · `bun` (newer)

PACKAGES

- ✓ `npm install`
- ✓ `package.json`
- ✓ `node_modules/` (ignore!)

DEBUG

- ✓ `console.log // fine`
- ✓ `console.table`
- ✓ editor breakpoints
- ✓ devtools "Sources"

you'll learn the rest by needing it. don't try to learn it all today.

WHAT'S NEXT

Build small things. Read other people's code.

Modern JS is huge. You only need a slice for daily work —
practice that slice.

PRACTICE

- ✓ rebuild a small UI you use daily
- ✓ solve 5 array-method problems
- ✓ fetch from a public API + render it

REFERENCE

- ✓ MDN — the JS reference
- ✓ javascript.info — deep tutorials
- ✓ You Don't Know JS (free, online)

questions →